Assignment 1

CS432: DATABASES

TEAM NAME - SCALEOPS
PROJECT - QUICKBITES

BHAVYA PARMAR
23110059

MOHIT
23110207

SONAGIRI THOSHITH KAUTILYA
23110316

SURRIYA GOKUL
23110324

TANISHQ BHUSHAN CHAUDHARI
23110329

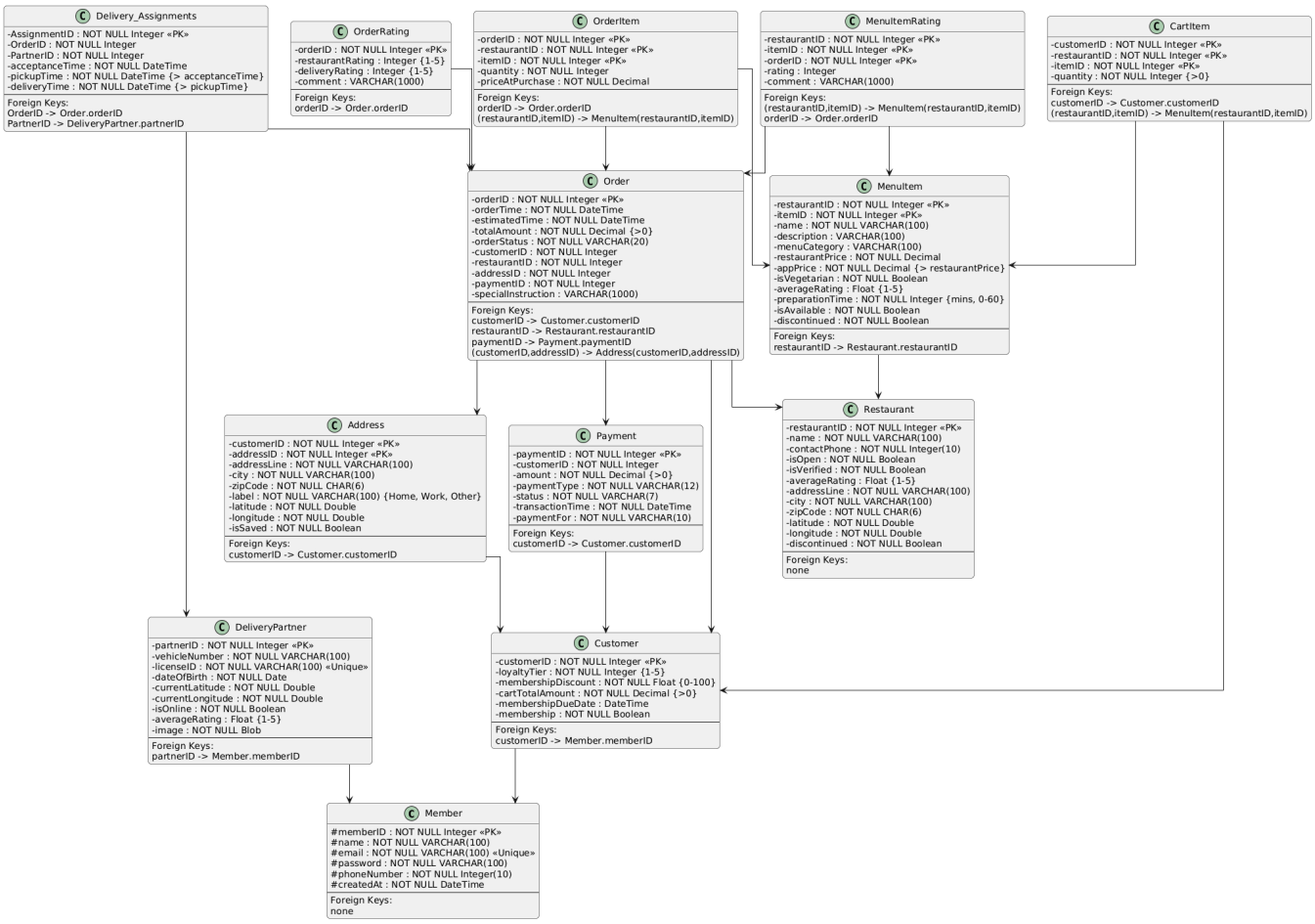
GitHub Repository

TABLE OF CONTENTS

MODULE A.....2
Relational Schema.....2
SQL Dump.....2
CONSTRAINTS AND REQUIREMENTS.....2
MODULE B.....4
ER DIAGRAM.....4
ER DIAGRAM NOTATION.....4
RELATIONSHIP JUSTIFICATIONS.....5
ADDITIONAL CONSTRAINTS.....8
CONTRIBUTIONS.....10

MODULE A

Relational Schema



SQL Dump

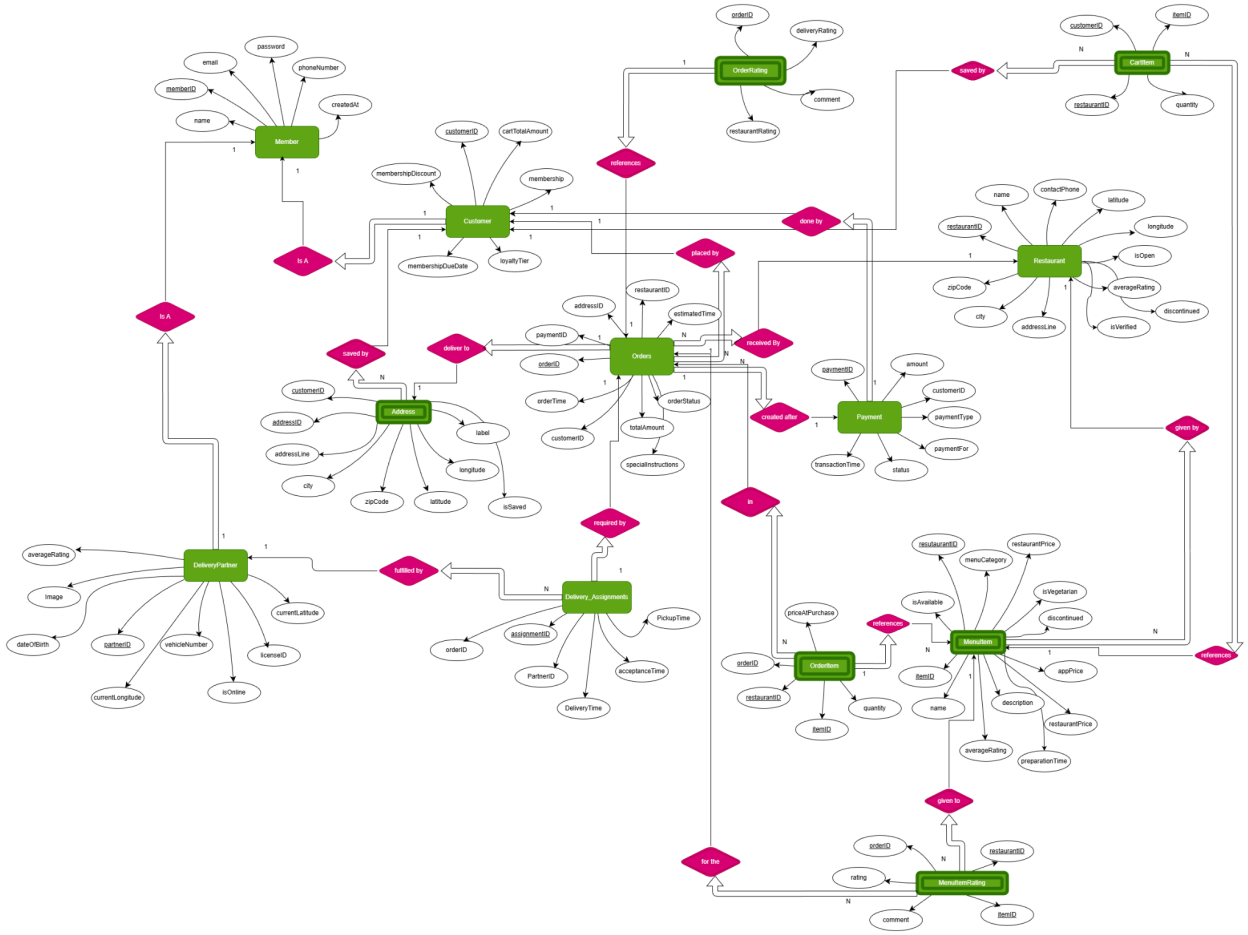
CONSTRAINTS AND REQUIREMENTS

- **Member table requirement** – A Member table was created with memberID, name, email, password, phoneNumber and createdAt. Instead of storing image and age directly in Member, those domain-specific attributes were placed in DeliveryPartner using image and dateOfBirth. This avoided unnecessary NULL values and kept the design normalized while still satisfying the constraint's intent.

- **Minimum functionalities** – The system supports more than five core functionalities, including user registration and role separation, browsing restaurants and menu items, cart management, order placement and tracking, payment handling, delivery assignment, and ratings/feedback, thereby meeting the requirement.
- **Minimum entities** – The schema contains well over 5 distinct entities covering users, restaurants, menus, orders, delivery, and payments; therefore, the constraint is satisfied.
- **Minimum tables** – The database contains 13 tables, exceeding the required minimum of 10.
- **Primary and foreign keys** – Every table has either a primary key or a composite primary key, and all relationships are enforced using foreign keys, ensuring proper linkage and unique identification.
- **Functional coverage** – All tables directly correspond to real features of a food ordering platform, and none are redundant, so the schema clearly supports the main domain functionalities.
- **Real-life data** – Each major table contains around 10–20 rows of meaningful synthetic data, such as realistic names, phone numbers, restaurant items, timestamps and locations, reflecting practical real-world scenarios.
- **Referential integrity** – All foreign keys use ON DELETE RESTRICT and ON UPDATE CASCADE, which prevents accidental deletion of dependent records and maintains consistency during updates.
- **NOT NULL columns** – Every table contains at least three NOT NULL attributes, as visible in the schema definitions, so the rule is adhered to.
- **Unique row identification** – Each table uses either a single primary key or a composite key, ensuring that every row is uniquely identified without ambiguity.
- **Logical constraints** – Domain logic is enforced using CHECK constraints, such as ensuring delivery time is after pickup time, ratings are within valid ranges, quantities are positive, and prices are comparable, ensuring real-world correctness.

MODULE B

ER DIAGRAM



ER DIAGRAM NOTATION

Entities (Green Rectangles)

- Represent real-world objects or concepts stored in the database.
- If Weak, then represented by a double-bordered box.

Relationships (Pink Diamonds)

- Represent associations between entities based on Foreign Keys.

- Direction doesn't represent Cardinality. It represents the Foreign Key direction.
- Double Arrow between entity and relationships represents Total Participation.

Attributes (White Ovals)

- Represent properties of entities or relationships.
- Connected directly to entities

Primary Keys

- Underlined attributes inside ovals.
- Uniquely identify each entity instance.

Cardinality Notation

- **1** → One
- **N** → Many
- Indicates relationship type:
    - **1: 1** → One-to-one
    - **1: N** → One-to-many
    - **N: M** → Many-to-many

RELATIONSHIP JUSTIFICATIONS

1. Member → Customer

- **Type:** One-to-One (1:1) [IS-A / Generalization]
- **Description:** Links the generic user credentials to the specific profile of a food-ordering customer.
- **Justification:** Specialized attributes like loyaltyTier are separated from common attributes like email to ensure normalization and role distinction.
- **Example:** Member Aman Shah (ID: 1) is linked to the Customer record (ID: 1).

2. Member → DeliveryPartner

- **Type:** One-to-One (1:1) [IS-A / Generalization]
- **Description:** Links the generic user credentials to the specific profile of a delivery driver.
- **Justification:** Separates logistics-specific data like licenseID and vehicleNumber from the base user table.
- **Example:** Member Driver One (ID: 11) is linked to the DeliveryPartner record (ID: 11).

3. Customer → Address

- **Type:** One-to-Many (1:N) [Weak Entity]
- **Description:** Associates a customer with multiple saved delivery locations (Home, Work).
- **Justification:** Modelled as a weak entity because an address (e.g., "Home") has no identity without the customer it belongs to.
- **Example:** Customer 1 has two addresses: AddressID 1 (Home) and AddressID 2 (Work).

4. Restaurant → MenuItem

- **Type:** One-to-Many (1:N) [Weak Entity]
- **Description:** Connects a restaurant entity to the specific food items it prepares.
- **Justification:** Items are existence-dependent on the restaurant; if the restaurant is removed, its unique menu must also be removed.
- **Example:** Restaurant Spice Garden (ID: 201) owns Paneer Butter Masala (Item: 1).

5. Customer → CartItem (Associative)

- **Type:** One-to-Many (1:N)
- **Description:** Links a customer to the temporary items currently held in their shopping cart.
- **Justification:** Allows a customer to hold multiple items in a "pending" state before an order is finalized.
- **Example:** Customer 1 has 2 units of Item 1 and 3 units of Item 2 in their cart.

6. MenuItem → CartItem (Associative)

- **Type:** One-to-Many (1:N)
- **Description:** Links a specific menu item to a customer's cart entry.
- **Justification:** Resolves the Many-to-Many relationship between Customer and MenuItems for the shopping cart phase.
- **Example:** Paneer Butter Masala (Item 1) appears in the carts of both Customer 1 and Customer 2.

7. Customer → Orders

- **Type:** One-to-Many (1:N)
- **Description:** Tracks the history of all orders placed by a specific customer account.
- **Justification:** Essential for account history; a customer places many orders, but an order belongs to only one customer.
- **Example:** Customer 3 placed Order #5003 and Order #5011.

8. Restaurant → Orders

- **Type:** One-to-Many (1:N)

- **Description:** Associates a specific order with the restaurant fulfilling it.
- **Justification:** Ensures every order is routed to the correct kitchen for preparation.
- **Example:** Order #5001 is assigned to Restaurant Spice Garden (ID: 201).

9. Address → Orders

- **Type:** One-to-Many (1:N)
- **Description:** Identifies which specific saved address was selected for a particular order's delivery.
- **Justification:** Locks the delivery location at the time of order to a specific address ID from the customer's profile.
- **Example:** Order #5004 was delivered to Customer 4's Address 2 (Work).

10. Payment → Orders

- **Type:** One-to-One (1:1)
- **Description:** Links a completed order to its specific financial transaction record.
- **Justification:** Ensures auditability by enforcing that every order has exactly one distinct payment record.
- **Example:** Order #5001 corresponds to Payment transaction #1001.

11. Orders → OrderItem (Associative)

- **Type:** One-to-Many (1:N)
- **Description:** Breaks down an order into its constituent line items (food, quantity, price).
- **Justification:** Resolves the Many-to-Many relationship between Orders and MenuItems, capturing the price *at the moment of purchase*.
- **Example:** Order #5001 contains 1 Paneer Butter Masala and 3 Garlic Naans.

12. MenuItem → OrderItem (Associative)

- **Type:** One-to-Many (1:N)
- **Description:** A specific menu item is referenced in an order line item.
- **Justification:** Allows tracking of which specific products are being sold across different orders.
- **Example:** Garlic Naan (Item 2) is listed in both Order #5001 and Order #5003.

13. Orders → Delivery_Assignments

- **Type:** One-to-One (1:1)
- **Description:** Connects an active order to its logistics and tracking details.
- **Justification:** Separates the static order details from the dynamic delivery lifecycle (pickup/drop times).
- **Example:** Order #5001 is linked to Assignment #9001.

14. DeliveryPartner → Delivery_Assignments

- **Type:** One-to-Many (1:N)
- **Description:** Assigns a specific driver to a delivery task.
- **Justification:** A partner completes many deliveries over time, but handles one assignment at a specific instance.
- **Example:** Partner 11 is assigned to deliver Order #5001 via Assignment #9001.

15. Orders → OrderRating

- **Type:** Zero-to-One (0..1) [Weak Entity]
- **Description:** Stores the feedback and score provided by the customer for a completed order.
- **Justification:** Optional relationship where the rating cannot exist without the order, and only one rating per order is allowed.
- **Example:** Order #5001 has a 5-star rating; Order #5004 (Cancelled) has no rating entry.

16. Orders → MenuItemRating

- **Type:** One-to-Many (1:N)
- **Description:** Links an order to specific feedback provided for individual items within that order.
- **Justification:** Allows granular feedback (e.g., "The curry was good, but the bread was cold") linked to the transaction.
- **Example:** Order #5001 has separate ratings for Paneer (5 stars) and Naan (4 stars).

17. MenuItem → MenuItemRating

- **Type:** One-to-Many (1:N)
- **Description:** Aggregates all ratings received for a specific menu item over time.
- **Justification:** Essential for calculating the average rating of a specific dish across all orders.
- **Example:** Paneer Butter Masala (Item 1) has ratings from Order #5001, #5003, and #5005.

ADDITIONAL CONSTRAINTS

1. Rating Scale Enforcement

- **Type:** Domain Constraint (Range)
- **Description:** Ensures all feedback scores for partners, restaurants, and items remain within a standard 1-to-5 star scale.
- **Justification:** Prevents invalid outliers (e.g., a rating of 0 or 10) that would skew aggregate calculations.

- **SQL Implementation:** CHECK (averageRating BETWEEN 1 AND 5) applied to Restaurant, DeliveryPartner, and OrderRating.

2. Delivery Timeline Logic

- **Type:** Temporal Consistency Constraint
- **Description:** Enforces a strictly linear progression of time for delivery milestones.
- **Justification:** Logically, a driver cannot deliver an order before picking it up, nor pick it up before accepting the assignment.
- **SQL Implementation:** CHECK (pickupTime > acceptanceTime) and CHECK (deliveryTime > pickupTime) in the Delivery_Assignments table.

3. Price Markup Logic

- **Type:** Business Logic Constraint
- **Description:** Mandates that the price listed on the app must be strictly higher than the base restaurant price.
- **Justification:** Ensures the platform's revenue model (markup) is always preserved and prevents data entry errors where the app price is lower than the cost.
- **SQL Implementation:** CHECK (appPrice > restaurantPrice) in the MenuItem table.

4. Financial Non-Negativity

- **Type:** Domain Constraint (Value)
- **Description:** Prevents negative values for monetary fields like cart totals, order amounts, and payments.
- **Justification:** Financial transactions cannot be negative in a purchase context; this prevents calculation errors.
- **SQL Implementation:** CHECK (totalAmount > 0) and CHECK (cartTotalAmount >= 0) across Orders, Cart, and Payment tables.

5. Order Status Workflow

- **Type:** State Transition Constraint
- **Description:** Restricts the orderStatus to a predefined set of valid lifecycle states.
- **Justification:** Prevents the system from entering an undefined state (e.g., "Shipped" instead of "OutForDelivery"), which would break application logic.
- **SQL Implementation:** CHECK (orderStatus IN ('Created', 'Preparing', 'ReadyForPickup', 'OutForDelivery', 'Delivered')).

6. Payment & Transaction Validity

- **Type:** specialized Domain Constraint
- **Description:** Restricts payment methods and statuses to specific valid types.

- **Justification:** Ensures that the system only processes supported payment modes and valid transaction outcomes.
- **SQL Implementation:** CHECK (paymentType IN ('OnQuickBites', 'COD')) and CHECK (status IN ('Pending', 'Success', 'Failed')).

7. Identity Uniqueness

- **Type:** Entity Integrity Constraint
- **Description:** Enforces strict uniqueness for critical identifiers beyond the Primary Key.
- **Justification:** A user cannot register multiple accounts with the same email, and a driver cannot share a license ID, preventing identity fraud.
- **SQL Implementation:** UNIQUE constraint on Member(email) and DeliveryPartner(licenseID).

8. Referential Safety (Delete Restriction)

- **Type:** Referential Integrity Constraint
- **Description:** Prevents the deletion of parent records (e.g., Customers, Restaurants) if dependent child records (Orders, Menu Items) exist.
- **Justification:** Preserves historical data integrity; a customer with past orders cannot be deleted outright, as that would orphan the order history.
- **SQL Implementation:** ON DELETE RESTRICT clause applied to all Foreign Key definitions.

CONTRIBUTIONS

MEMBER	CONTRIBUTIONS
BHAVYA PARMAR (23110059)	Module A - Database Implementation on MySQL Workbench based on the designed database schema.
MOHIT (23110207)	Module A - Schema design using PlantUML Editor, constraint definition, and overall database structure planning.
SONAGIRI THOSHITH KAUTILYA (23110316)	Module B - Responsible for system design analysis and managing the precise mapping of attributes to the ER schema.
SURRIYA GOKUL (23110324)	Module B - Responsible for end-to-end conceptual modelling, designing the comprehensive ER diagram
TANISHQ BHUSHAN CHAUDHARI (23110329)	Module A - Schema design using PlantUML Editor, constraint definition, and overall database structure planning.